

AI ANPR Architecture Milestones

Version: 1.0

Status: Planned

Implementation Strategy: Architecture-by-Architecture

1. Overview

This roadmap implements the AI ANPR system architecture incrementally rather than following the original implementation phases.

The objective is to build the ANPR platform in layers:

```
Foundation
→ Configuration
→ Source Processing
→ Detection
→ OCR
→ Tracking
→ Event Generation
→ Backend Integration
→ Frontend Integration
→ Realtime Runtime
→ Testing
→ Optimization
```

This approach ensures:

- Stable architecture before feature expansion.
 - Early validation of each layer.
 - Reduced integration risk.
 - Clear acceptance criteria.
 - Compatibility with existing Laravel backend.
 - Compatibility with existing React frontend architecture.
-

2. Architecture Principles

The implementation follows the ANPR design document principles:

```
Load models once
Open source once
Keep runtime state in memory
```

```
Save only useful evidence
Create one event per vehicle
Send backend work asynchronously
Backend owns final records
```

The first version intentionally remains small:

```
main.py
config.py
anpr.py
backend.py
```

Do not split into many runtime modules prematurely.

Milestone 0 — Project Foundation Architecture

Goal

Create the initial AI ANPR project shell and establish development conventions.

0.1 Folder Structure

Scope

Create:

```
new-ai-anpr/
├── README.md
├── requirements.txt
├── .env.example
├── .gitignore
├── main.py
├── config.py
├── anpr.py
├── backend.py
├──
├── models/
│   ├── vehicle/
│   └── plate/
├──
├── samples/
│   ├── videos/
│   └── images/
├──
```

```
| runs/
|
| .cache/
```

Passing Criteria

- Folder structure matches architecture document.
 - `.pt` files ignored by Git.
 - `runs/` ignored.
 - `.cache/` ignored.
-

0.2 Runtime Skeleton

Scope

Create:

```
main.py
config.py
anpr.py
backend.py
```

Passing Criteria

- Files compile successfully.
 - Clear responsibility separation.
 - No unnecessary modules created.
-

0.3 Development Baseline

Scope

Create:

- setup instructions
- install instructions
- run instructions

Passing Criteria

Developer can:

```
pip install -r requirements.txt
python main.py check-config
```

0.4 Output Structure

Scope

Define:

```
runs/run_YYYYMMDD_HHMMSS/  
├─ worker.log  
├─ worker_summary.json  
└─ events.jsonl
```

Passing Criteria

Dry execution creates run directory.

Milestone Pass Condition

Project can be cloned and initialized without backend dependency.

Milestone 1 — Configuration and CLI Architecture

Goal

Build a stable entry point for all future runtime operations.

1.1 Typed Configuration Loader

Scope

Load:

```
ANPR_SOURCE  
ANPR_RTSP_URL  
ANPR_VIDEO_PATH  
ANPR_IMAGE_PATH  
ANPR_CAMERA_INDEX  
ANPR_VEHICLE_MODEL  
ANPR_PLATE_MODEL  
ANPR_DEVICE
```

```
ANPR_BACKEND_ENABLED
...
```

Passing Criteria

Invalid configuration fails immediately.

1.2 Source Validation

Scope

Validate:

- RTSP URL
- Video path
- Image path
- Model files
- Backend credentials
- Camera UUID

Passing Criteria

Configuration errors clearly reported.

1.3 CLI Commands

Scope

Support:

```
python main.py check-config

python main.py run --source rtsp

python main.py run --source video

python main.py run --source image

python main.py run --source webcam

python main.py run --source-path <path>

python main.py flush-backend-queue
```

Passing Criteria

All commands parse correctly.

1.4 Runtime Summary Contract

Scope

Define:

```
{  
  "status": "",  
  "frames_read": 0,  
  "frames_processed": 0,  
  "events_finalized": 0  
}
```

Passing Criteria

Summary generated for every run.

Milestone Pass Condition

CLI and configuration become stable public interfaces.

Milestone 2 — Source Reader and Frame Scheduler Architecture

Goal

Create unified source handling.

2.1 Source Resolution

Scope

Support:

```
RTSP  
Video  
Image  
Webcam
```

Passing Criteria

Correct source handler selected automatically.

2.2 FramePacket Contract

Scope

Create:

```
@dataclass
class FramePacket:
    frame_index: int
    timestamp: float
    image: np.ndarray
    source_type: str
    source_path: str | None
    is_last: bool
```

Passing Criteria

All runtime processing uses FramePacket.

2.3 Frame Scheduler

Scope

Implement:

```
ANPR_TARGET_FPS
```

Frame skipping.

Passing Criteria

30 FPS video processes at configured target FPS.

2.4 Source End Detection

Scope

Detect:

- image complete

- video complete
- stream stop

Passing Criteria

Finalization can occur correctly.

2.5 Frame Metrics

Scope

Track:

- frames read
- frames processed

Passing Criteria

Metrics accurate.

Milestone Pass Condition

All sources produce FramePackets consistently.

Milestone 3 — Model Loading and Detector Architecture

Goal

Implement detection architecture.

3.1 Vehicle Model Loader

Scope

Load YOLO vehicle detector once.

Passing Criteria

Single startup load.

3.2 Plate Model Loader

Scope

Load:

```
models/plate/license-plate-finetune-v1s.pt
```

Passing Criteria

Model loaded successfully.

3.3 Device Selection

Scope

Support:

```
ANPR_DEVICE=cpu  
ANPR_DEVICE=cuda
```

Passing Criteria

Runtime uses configured device.

3.4 Vehicle Detection Wrapper

Scope

Create:

```
detect_vehicles()
```

Passing Criteria

Returns normalized detection results.

3.5 Plate Detection Wrapper

Scope

Create:

```
detect_plates()
```

Passing Criteria

Returns normalized plate detections.

3.6 Detection Metrics

Scope

Track:

```
vehicle detection ms  
plate detection ms
```

Passing Criteria

Metrics recorded.

Milestone Pass Condition

Vehicle and plate detections operate independently.

Milestone 4 — OCR and Plate Normalization Architecture

Goal

Convert plate crops into valid Malaysian plate candidates.

4.1 Plate Crop Extraction

Passing Criteria

Valid crops generated.

4.2 Plate Preprocessing

Scope

Optional:

```
grayscale  
resize  
denoise  
sharpen
```

Passing Criteria

OCR input quality improved.

4.3 OCR Engine Wrapper

Scope

Create:

```
read_plate_text()
```

Passing Criteria

Returns text + confidence.

4.4 Plate Normalization

Scope

Convert:

```
abc 1234  
→  
ABC1234
```

Passing Criteria

Normalized consistently.

4.5 Plate Validation

Scope

Reject:

- empty
- invalid length
- invalid format

Passing Criteria

Only valid candidates enter voting.

4.6 OCR Metrics

Passing Criteria

Average OCR timing recorded.

Milestone Pass Condition

System generates normalized plate candidates.

Milestone 5 — Tracking and Vote Buffer Architecture

Goal

Generate one final event per vehicle.

5.1 TrackState Model

Scope

TrackState

stores:

- bbox
- votes
- evidence

- timestamps

Passing Criteria

Tracks persist across frames.

5.2 IoU Tracking

Passing Criteria

Vehicle retains same track id.

5.3 Vote Buffer

Scope

Store:

- plate text
- confidence
- timestamp

Passing Criteria

Multiple OCR candidates accumulated.

5.4 Best Evidence State

Scope

Store:

```
best full frame
best plate crop
best annotated frame
```

Passing Criteria

Evidence available at finalization.

5.5 Track Expiry

Scope

Use:

```
ANPR_TRACK_EXPIRY_SECONDS
```

Passing Criteria

Vehicle disappearance triggers finalization.

5.6 Source End Finalization

Passing Criteria

Video/image flushes active tracks.

5.7 Early Finalization

Scope

Use:

```
ANPR_EARLY_FINALIZE_MIN_VOTES  
ANPR_EARLY_FINALIZE_MIN_CONFIDENCE
```

Passing Criteria

High-confidence plate finalizes early.

Milestone Pass Condition

One vehicle results in one finalized candidate.

Milestone 6 — Final Event and Evidence Architecture

Goal

Create finalized event records and evidence.

6.1 FinalizedEvent Contract

Scope

Create:

```
FinalizedEvent
```

Passing Criteria

Consistent event structure.

6.2 Evidence Layout

Scope

```
evidence/  
├─ full  
├─ plate  
└─ annotated
```

Passing Criteria

All evidence files saved.

6.3 Annotated Evidence

Scope

Draw:

- vehicle box
- plate box
- confidence
- plate text

Passing Criteria

Readable annotations.

6.4 JSONL Writer

Scope

Append:

```
events.jsonl
```

Passing Criteria

Valid JSONL output.

6.5 Duplicate Prevention

Scope

Optional:

```
ANPR_DUPLICATE_COOLDOWN_SECONDS
```

Passing Criteria

Duplicates suppressed.

6.6 Dry Run Support

Passing Criteria

No backend side effects.

Milestone Pass Condition

Dry-run generates:


```
events.jsonl  
evidence  
worker_summary.json
```

Milestone 7 — Backend Client and Queue Architecture

Goal

Integrate backend safely.

7.1 Token Cache

Scope

```
.cache/backend_token.json
```

Passing Criteria

Token reused.

7.2 Login and Refresh

Passing Criteria

401 automatically refreshes token.

7.3 Queue File

Scope

```
.cache/backend_queue.jsonl
```

Passing Criteria

Events queued asynchronously.

7.4 Event Posting

Payload

```
{
  "camera_id": "",
  "plate_number": "",
  "confidence": 0.0,
  "detection_time": "",
  "is_valid": true
}
```

Passing Criteria

Backend creates ANPR event.

7.5 Retry Handling

Passing Criteria

Retries obey configured limit.

7.6 Queue Flush Command

Passing Criteria

Pending jobs can be resent.

Milestone Pass Condition

Backend event posting is reliable and non-blocking.

Milestone 8 — Backend ANPR Data Architecture Alignment

Goal

Align AI outputs with Laravel ANPR modules.

8.1 Camera Mapping

Passing Criteria

Configured camera UUID exists.

8.2 ANPR Event Contract

Passing Criteria

AI payload accepted by backend.

8.3 Vehicle Strategy

Passing Criteria

Unknown vehicles still create events.

8.4 Image Metadata Integration

Passing Criteria

Backend image records created.

8.5 Event Logs Integration

Passing Criteria

Backend logs created.

8.6 Response Handling

Passing Criteria

Validation and API errors handled gracefully.

Milestone Pass Condition

Backend stores:

ANPR Event
ANPR Images
ANPR Event Logs

for every finalized detection.

Milestone 9 — Evidence Delivery Architecture

Goal

Deliver evidence to backend.

9.1 Metadata Mode

Passing Criteria

Image metadata records created.

9.2 Image Root Compatibility

Passing Criteria

Backend can resolve local evidence.

9.3 Evidence Logging

Passing Criteria

Success/failure visible in logs.

9.4 Upload Mode Design

Passing Criteria

Future upload contract documented.

9.5 Retention Policy

Passing Criteria

Local evidence retention configurable.

Milestone Pass Condition

Backend event detail references all evidence.

Milestone 10 — Frontend ANPR Feature Architecture

Goal

Expose ANPR events through React frontend.

10.1 Feature Module

Scope

`feature/anpr-monitoring/`

Passing Criteria

Matches existing feature structure.

10.2 Datasource Layer

Passing Criteria

API integration implemented.

10.3 Repository Layer

Passing Criteria

Response normalization implemented.

10.4 Controller Layer

Passing Criteria

State management isolated.

10.5 Event List Page

Passing Criteria

Displays detections.

10.6 Event Detail Page

Passing Criteria

Displays evidence and logs.

10.7 Routing and Permissions

Passing Criteria

Protected routes work.

10.8 UI Convention

Passing Criteria

Uses project UI patterns.

Milestone Pass Condition

Frontend displays ANPR detections end-to-end.

Milestone 11 — Realtime RTSP Runtime Architecture

Goal

Production-ready live runtime.

11.1 Continuous RTSP Processing

Passing Criteria

Processes stream continuously.

11.2 Reconnection Handling

Passing Criteria

Temporary failures recover automatically.

11.3 Queue-safe Runtime

Passing Criteria

Backend failures do not stop detection.

11.4 Runtime Health Logs

Passing Criteria

Operational metrics logged.

11.5 Runtime Summary

Passing Criteria

Summary generated after shutdown.

11.6 Graceful Shutdown

Passing Criteria

Active tracks finalized.

Milestone Pass Condition

Stable RTSP deployment possible.

Milestone 12 — Testing Architecture

Goal

Protect core logic from regressions.

12.1 Unit Tests

Areas

- config
- normalization
- voting
- tracking
- queue
- token cache

Passing Criteria

All pass.

12.2 Integration Tests

Areas

- image flow
- video flow
- backend flow
- queue retry

Passing Criteria

All pass.

12.3 Manual Tests

```
python main.py check-config

python main.py run --source image --dry-run

python main.py run --source video --dry-run

python main.py run --source rtsp --dry-run
```



```
python main.py flush-backend-queue
```

Passing Criteria

All commands operate successfully.

Milestone Pass Condition

System fully testable.

Milestone 13 — Performance and Accuracy Tuning Architecture

Goal

Improve runtime quality after correctness is achieved.

13.1 FPS Tuning

Passing Criteria

3-5 FPS achieved.

13.2 Confidence Tuning

Passing Criteria

False positives reduced.

13.3 Tracker Tuning

Passing Criteria

One event per vehicle.

13.4 Duplicate Cooldown

Passing Criteria

Spam events reduced.

13.5 OCR Optimization

Passing Criteria

OCR calls minimized.

13.6 CPU vs GPU Evaluation

Passing Criteria

Recommended deployment profile documented.

13.7 Optional ONNX Support

Passing Criteria

Alternative runtime path documented.

Milestone Pass Condition

Performance targets achieved:

Metric	Target
FPS	3-5
Event Latency	2-5 sec
Backend Posting	Async
Model Loading	Once
OCR Calls	Minimized

Final Delivery Criteria

The AI ANPR platform is considered complete when:

RTSP / Video / Image / Webcam



Vehicle Detection



Plate Detection



OCR



Vote Buffer



Track Finalization



Evidence Creation



Backend Queue



Laravel ANPR Event



Laravel ANPR Images



Laravel ANPR Logs



React Dashboard Display

All milestones passed.

All acceptance criteria satisfied.

End of document.